Roteamento Dinâmico de Consultas SQL Baseado em Metadados de Tabelas Apache Iceberg*

Dynamic SQL Query Routing Based on Apache Iceberg Table Metadata

Aristides Henrique Gonçalves da Cruz[1]
Gustavo Luís Soares[2]

Resumo

O crescimento do volume de dados em *data lakes* tem impulsionado a necessidade de arquiteturas adaptativas que equilibrem custo e *performance* na execução de consultas SQL. Este trabalho apresenta o *DyraSQL*, um *framework* para roteamento dinâmico de consultas SQL baseado em metadados de tabelas *Apache Iceberg*, obtidos por meio do comando *EXPLAIN (TYPE IO)* do *Trino*. O sistema direciona automaticamente consultas para *clusters* de processamento com capacidades distintas, classificados como *small*, *medium* e *large*, a partir de um algoritmo de pontuação que combina volume de dados, complexidade da consulta e histórico de execuções. A metodologia envolve extração de metadados sobre volume efetivo e custos de I/O, análise sintática da consulta SQL e reaproveitamento de decisões em *cache* temporal no PostgreSQL. O protótipo foi avaliado em ambiente local orquestrado por Docker Compose, com tabelas *Apache Iceberg* geradas sobre armazenamento de objetos MinIO e catálogo REST, demonstrando que o sistema calcula *scores* consistentes e classifica consultas nas faixas de capacidade definidas. Os resultados indicam que o roteamento baseado em metadados contribui para o uso mais eficiente de recursos de processamento, oferecendo um modelo adaptativo aplicável a *clusters Trino* locais e, em trabalhos futuros, a ambientes de nuvem.

Palavras-chave: Consultas SQL. Trino. Apache Iceberg. Data lake. Trino Gateway.

*Trabalho de conclusão de curso, Sistemas de Informação, Unidade São Gabriel

[1]Engenheiro de Dados, Sistemas de Informação, Brasil– arihenriquedev@hotmail.com
[2]Orientador, Instituto de Ciências Exatas e de Informática da PUC Minas, Brasil– gsoares@pucminas.br

Abstract

The growth of data volume in data lakes has increased the need for adaptive architectures that balance cost and performance in SQL query execution. This work presents DyraSQL, a framework for dynamic SQL query routing based on Apache Iceberg table metadata obtained through Trino's EXPLAIN (TYPE IO) command. The system automatically directs queries to processing clusters with distinct capacities, classified as small, medium, and large, using a scoring algorithm that combines data volume, query complexity, and execution history. The methodology extracts metadata about effective data volume and I/O costs, performs syntactic analysis of SQL queries, and reuses decisions through a temporal cache in PostgreSQL. The prototype was evaluated in a local Docker Compose environment, with generated Apache Iceberg tables on MinIO object storage and a REST catalog, showing that the system computes consistent scores and classifies queries into the defined capacity ranges. The results indicate that metadata-based routing contributes to more efficient use of processing resources, offering an adaptive model for local Trino clusters and, as future work, for cloud environments.

Keywords: SQL queries. Trino. Apache Iceberg. Data lake. Trino Gateway.

1 INTRODUÇÃO

O crescimento do volume de dados tem impulsionado a adoção de arquiteturas escaláveis para análise e tomada de decisão (IBM, 2025). Nesse contexto, o desafio não está apenas em armazenar grandes volumes de dados de forma eficiente, mas também em processá-los equilibrando custos e desempenho (WEINTRAUB *et al.*, 2024). A arquitetura de *data lakes* baseada em armazenamento de objetos oferece escalabilidade e custos reduzidos, sendo que a separação entre camadas de armazenamento e processamento permite utilização de recursos computacionais em modo *on-demand* (DREMIO, 2025; WEINTRAUB *et al.*, 2024).

Ferramentas como o *Apache Iceberg* têm se destacado ao permitir a gestão eficiente de tabelas em *data lakes*, oferecendo versionamento, *schema evolution* e melhorias de consulta (Apache Software Foundation, 2025). Mecanismos de execução distribuída, como o *Trino*, viabilizam consultas de alta *performance* sobre dados armazenados em sistemas de objetos, por meio de processamento distribuído entre múltiplos nós trabalhadores (TrinoDB, 2025). No entanto, a escolha do ambiente de execução das consultas ainda tende a ser estática, sendo que sistemas tradicionais não adaptam automaticamente a infraestrutura de processamento às características específicas de cada consulta (WEINTRAUB *et al.*, 2024; XUE *et al.*, 2024). Trabalhos recentes têm explorado roteamento baseado em modelos de custo aprendidos (STRAUSZ; PARDON; GIURGIU, 2025) e elasticidade em tempo de execução para análise de dados em nuvem (ZHANG; ZHANG; MENG, 2025).

A pesquisa propõe o *DyraSQL*, uma abordagem adaptativa para direcionamento de consultas SQL, considerando características previamente disponíveis nos metadados. A proposta consiste em analisar a complexidade das consultas e o volume efetivo de dados antes da execução e, a partir disso, direcioná-las para *clusters Trino* de capacidades distintas, classificados como *small*, *medium* e *large*. Tal abordagem busca viabilizar uma utilização mais eficiente dos recursos disponíveis, equilibrando custo e *performance*. Xue et al. (2024) demonstram a importância de execução adaptativa de consultas em ambientes de *data lakehouse* (XUE *et al.*, 2024), reforçando a relevância desta pesquisa.

A capacidade de direcionar automaticamente consultas para o ambiente mais adequado resulta em melhor utilização de recursos computacionais e em redução de provisionamento excessivo para consultas leves. Este trabalho desenvolve o *DyraSQL*, um mecanismo de decisão que utiliza informações de metadados para definir, de forma automática, o *cluster* mais adequado para a execução de consultas SQL. Para alcançar este propósito, o trabalho mapeia os metadados disponíveis em tabelas *Apache Iceberg*, identificando quais podem ser utilizados como indicadores de volume de dados, complexidade da consulta e custo esperado de execução. Estabelece regras de decisão para roteamento, definindo parâmetros que permitam classificar consultas nas três faixas de capacidade. Projeta e desenvolve um protótipo funcional do *DyraSQL* em ambiente local, utilizando o *Trino* como *engine* de consultas, Docker Compose para orquestração, MinIO para armazenamento de objetos e PostgreSQL para *cache* e histórico. Realiza experimentos controlados nesse ambiente, a fim de verificar a viabilidade técnica

do *DyraSQL* e coletar métricas de consistência dos *scores*, latência do *EXPLAIN (TYPE IO)* e comportamento do *cache*. Compara o comportamento do roteamento dinâmico com a lógica estática de um único *cluster*, avaliando a capacidade do algoritmo de diferenciar consultas leves, intermediárias e pesadas.

Este trabalho está organizado da forma apresentada. A Seção 2 aborda o referencial teórico. A Seção 3 apresenta a revisão bibliográfica. A Seção 4 detalha a metodologia. A Seção 5 apresenta os experimentos e resultados. A Seção 6 apresenta as considerações finais e trabalhos futuros.

2 REFERENCIAL TEÓRICO

Esta seção apresenta os conceitos fundamentais que embasam a pesquisa proposta, abordando arquiteturas de *data lakes*, análise de dados em *big data*, tecnologias de processamento distribuído e o roteamento de consultas utilizado no desenvolvimento do *DyraSQL*. Os tópicos são organizados de forma a estabelecer o contexto teórico necessário para compreensão da solução proposta.

2.1 Arquiteturas de *Data Lake*

Data lakes representam repositórios centralizados que armazenam grandes volumes de dados em seu formato bruto, sem estruturação prévia, permitindo armazenamento escalável e flexível para diferentes tipos de análises (DREMIO, 2025). Diferentemente de *data warehouses*, que organizam dados em esquemas estruturados pré-definidos, *data lakes* mantêm dados em formato nativo, facilitando a ingestão de dados diversos e a exploração analítica posterior (DREMIO, 2025). Esta arquitetura é particularmente adequada para ambientes de *big data*, nos quais o volume, variedade e velocidade dos dados tornam a estruturação prévia impraticável ou limitante (IBM, 2025).

A arquitetura de *data lakes* baseada em armazenamento de objetos oferece escalabilidade e custos reduzidos de armazenamento, sendo que os dados são organizados em *buckets* e objetos, permitindo acesso direto através de APIs (DREMIO, 2025). Esta abordagem elimina a necessidade de provisionamento prévio de capacidade, permitindo que organizações armazenem grandes volumes de dados de forma econômica (IBM, 2025). O *DyraSQL* aproveita esta arquitetura para acessar metadados de tabelas *Apache Iceberg* armazenadas em *data lakes*, utilizando informações sobre volume e estrutura dos dados para tomada de decisões de roteamento. No protótipo local, o armazenamento de objetos é provido pelo MinIO, que expõe uma API compatível com sistemas de objetos utilizados em *data lakes*.

2.2 Análise de Dados em *big data*

A análise de dados em *big data* envolve o processamento de volumes massivos de informações que excedem a capacidade de sistemas tradicionais de banco de dados, requerendo arquiteturas distribuídas e técnicas especializadas de processamento (IBM, 2025). Os desafios principais incluem o volume dos dados, a variedade de formatos e fontes, a velocidade de geração e a necessidade de extrair valor em tempo hábil para tomada de decisão (IBM, 2025). Sistemas de processamento distribuído, como o *Trino*, viabilizam a análise de grandes volumes de dados através de paralelização e distribuição de carga de trabalho entre múltiplos nós computacionais (TrinoDB, 2025).

A eficiência na análise de *big data* depende da escolha adequada da arquitetura de processamento, sendo que consultas leves podem ser executadas em *clusters* menores, enquanto consultas complexas requerem infraestrutura com maior capacidade computacional (IBM, 2025). O *DyraSQL* aborda este desafio através de roteamento dinâmico que direciona consultas para o *cluster* mais adequado baseado em características dos dados e complexidade das operações, melhorando o custo-benefício do processamento.

2.3 *Trino*

O *Trino* é um *engine* de consultas SQL distribuído projetado para consultar grandes volumes de dados armazenados em diversos sistemas, incluindo *data lakes* baseados em armazenamento de objetos (TrinoDB, 2025). A arquitetura do *Trino* é baseada em um modelo de coordenação distribuída, sendo que um nó coordenador recebe consultas SQL e coordena a execução entre múltiplos nós trabalhadores (*workers*), cada um processando uma porção dos dados em paralelo (TrinoDB, 2025). Esta arquitetura permite escalabilidade horizontal através da adição de nós trabalhadores, aumentando a capacidade de processamento conforme a demanda cresce (TrinoDB, 2025).

O *Trino* suporta conectores para diversos sistemas de armazenamento, incluindo *Apache Iceberg*, permitindo consultas SQL unificadas sobre dados armazenados em diferentes formatos e localizações (TrinoDB, 2025). O *DyraSQL* utiliza o *Trino* como *engine* de consultas, aproveitando sua capacidade de processamento distribuído e o suporte nativo a tabelas *Apache Iceberg*. A integração com *Trino Gateway* permite roteamento de consultas para diferentes *clusters* de execução baseado em análise prévia de metadados (TRINODB, 2025). No ambiente experimental, o conector *Iceberg* é configurado com catálogo REST, de modo que múltiplos *clusters* compartilhem o mesmo catálogo e o mesmo *warehouse* de objetos.

2.4 Catálogo de Metadados

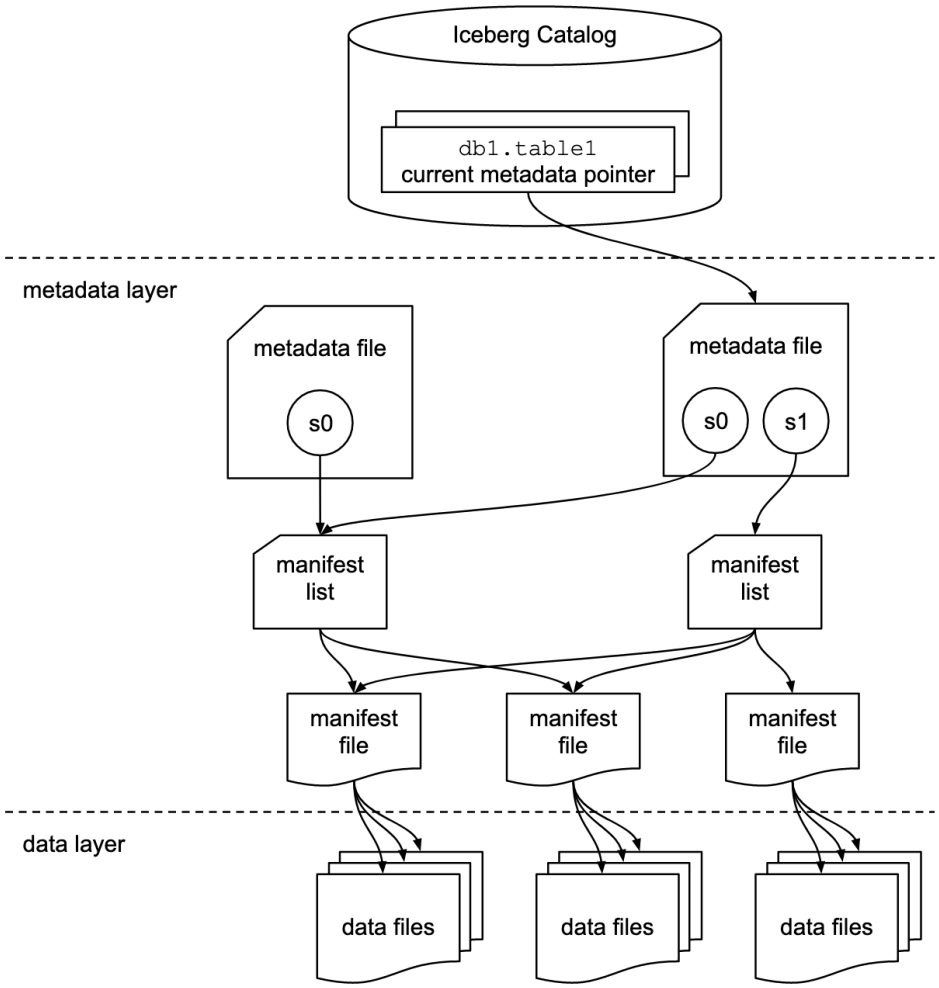
Sistemas de consulta sobre *data lakes* dependem de um catálogo que descreva esquemas, localização dos dados e estatísticas das tabelas, permitindo que o *engine* descubra e acesse os dados sem conhecer a estrutura física dos arquivos (Apache Software Foundation, 2025). O *Apache Iceberg* admite o catálogo REST, que expõe uma API HTTP para operações de metadados e é a implementação utilizada no protótipo.

O *DyraSQL* não depende de um provedor específico de catálogo. O protótipo local utiliza o catálogo REST do *Apache Iceberg* compartilhado pelos *clusters Trino*, garantindo consistência de metadados independentemente do *cluster* selecionado para execução.

2.5 *Apache Iceberg*

O *Apache Iceberg* é uma especificação de tabela para *data lakes* que oferece capacidades avançadas de gerenciamento de dados, incluindo versionamento, *schema evolution* e melhorias de consulta (Apache Software Foundation, 2025). Sua arquitetura permite o armazenamento eficiente de metadados que podem ser utilizados para melhoria de consultas e tomada de decisões sobre processamento (Apache Software Foundation, 2025). O *Apache Iceberg* mantém uma estrutura hierárquica de metadados que oferece informações detalhadas sobre as tabelas, sendo que a Figura 1 ilustra esta estrutura com os três níveis principais, arquivo de metadados (*metadata*.json), lista de manifestos (*manifest-list*) e arquivos de manifesto (*manifest*) (Apache Software Foundation, 2025). A especificação define que cada tabela possui um arquivo de metadados principal que contém informações sobre o esquema, partições, *snapshots* e histórico de alterações, sendo que os manifestos listam os arquivos de dados que compõem cada *snapshot*, incluindo estatísticas de colunas, contagens de registros e informações de partição (Apache Software Foundation, 2025). Esta arquitetura permite que sistemas de consulta baseiem a execução em informações precisas sobre o volume e a distribuição dos dados, sendo essa camada de estatísticas por arquivo a origem dos valores que o comando *EXPLAIN (TYPE IO)* do *Trino* expõe de forma agregada, como *outputSizeInBytes* e *outputRowCount*, consumidos pelo *DyraSQL* como entrada do Fator de volume descrito na Seção 4.4.

Figura 1 – Estrutura de Metadados do *Apache Iceberg*



Fonte: (Apache Software Foundation, 2025).

A Figura 1 demonstra a estrutura hierárquica de metadados do *Apache Iceberg*, evidenciando como os três níveis principais organizam informações sobre volume, estrutura e distribuição dos dados. Esta organização facilita a extração de metadados relevantes para tomada de decisões de roteamento no *DyraSQL*.

2.6 Processamento em *clusters*

A execução de consultas SQL em *data lakes* ocorre tipicamente em *clusters* de processamento com capacidades distintas de memória, paralelismo e tempo máximo de consulta. Consultas leves, com volume reduzido e poucas operações, podem ser atendidas por *clusters* menores, com menor consumo de recursos. Consultas intermediárias demandam paralelismo moderado. Consultas pesadas, com grande volume de dados ou elevada complexidade, requerem *clusters* mais robustos (TrinoDB, 2025; IBM, 2025).

O *DyraSQL* adota três faixas de capacidade, *cluster small*, *cluster medium* e *cluster*

large, implementadas no ambiente local como três instâncias *Trino* com limites distintos de memória e tempo de execução. A classificação em faixas permite que o algoritmo de pontuação selecione o ambiente antes da execução, evitando tanto a subutilização de infraestrutura robusta em consultas simples quanto a saturação de *clusters small* em consultas pesadas.

2.7 Roteamento Dinâmico de Consultas

O roteamento dinâmico de consultas envolve a seleção automática do ambiente de execução mais adequado baseado em características da consulta e dos dados, sendo uma abordagem explorada em diversos contextos para melhoria de recursos computacionais. Trabalhos recentes têm demonstrado a importância de execução adaptativa e robusta de consultas em ambientes de *data lakehouse* (XUE *et al.*, 2024), sendo que abordagens baseadas em modelos de custo aprendidos (STRAUSZ; PARDON; GIURGIU, 2025) e elasticidade em tempo de execução (ZHANG; ZHANG; MENG, 2025) têm sido exploradas para melhoria de custo-benefício. O *DyraSQL* implementa roteamento dinâmico baseado em análise prévia de metadados de tabelas *Apache Iceberg*, direcionando consultas para *clusters small*, *medium* ou *large* com base em volume de dados, complexidade da consulta e histórico de execuções similares.

3 REVISÃO BIBLIOGRÁFICA

Esta seção apresenta uma revisão de trabalhos relacionados ao roteamento dinâmico de consultas, melhoria de custo-benefício em ambientes de processamento e utilização de metadados para tomada de decisões em sistemas de dados. A revisão bibliográfica complementa o referencial teórico apresentado na Seção 2, focando em trabalhos que abordam problemas similares ao proposto neste trabalho e técnicas relacionadas utilizadas na literatura.

3.1 Trabalhos sobre Roteamento Dinâmico de Consultas

Xue et al. (2024) propuseram um sistema adaptativo que ajusta dinamicamente a estratégia de execução em ambientes de *lakehouse* em larga escala, conforme características da consulta e condições do ambiente (XUE *et al.*, 2024); o *DyraSQL* complementa essa abordagem através de análise prévia de metadados antes da execução, em vez de ajustes em tempo real.

Strausz, Pardon e Giurgiu (2025) propuseram um otimizador baseado em modelo de custo aprendido para *workloads* SQL *multi-engine*, prevendo por aprendizado de máquina o custo de execução em diferentes *engines* (STRAUSZ; PARDON; GIURGIU, 2025); o *DyraSQL* utiliza abordagem similar através do Fator histórico, mas foca em roteamento entre configurações de *clusters Trino*, e não entre *engines* distintas.

Zhang, Zhang e Meng (2025) exploraram elasticidade em tempo de execução, ajustando recursos computacionais durante a execução com base em métricas de *performance* em tempo real (ZHANG; ZHANG; MENG, 2025); o *DyraSQL* é complementar, pois realiza a análise prévia de metadados para seleção do *cluster* antes da submissão, e não ajustes durante a execução.

3.2 Trabalhos sobre Custos em *data lakes*

Weintraub et al. (2024) propuseram uma abordagem de melhoria de consultas em *data lakes* baseada em planos de cobertura balanceados, utilizando índices e metadados para reduzir o volume de dados via *data skipping* (WEINTRAUB *et al.*, 2024); o *DyraSQL* usa abordagem similar de análise de metadados do *Apache Iceberg*, mas para direcionar consultas entre *clusters*, e não para otimizar dentro de uma única arquitetura.

Ding et al. (2024) apresentaram um sistema de *layouts* de dados multidimensionais automatizados no Amazon Redshift, que usa aprendizado de máquina para organizar dados conforme padrões de consultas (DING *et al.*, 2024); o *DyraSQL* utiliza abordagem similar de aprendizado contínuo, ajustando pesos do modelo de decisão a partir do histórico de custo e *performance*, mas com foco em roteamento entre arquiteturas de processamento, e não em organização de dados.

3.3 Trabalhos sobre Utilização de Metadados

A utilização de metadados para melhoria de consultas tem sido amplamente explorada na literatura, sendo que sistemas como o *Apache Iceberg* oferecem estruturas hierárquicas de metadados para estimativa de volume, análise de partições e melhoria de consultas (Apache Software Foundation, 2025). Trabalhos anteriores demonstraram a eficácia de metadados para *data skipping* e melhoria de consultas (WEINTRAUB *et al.*, 2024); o *DyraSQL* estende essa abordagem para roteamento dinâmico entre diferentes *clusters*.

Sistemas de processamento distribuído como o *Trino* utilizam metadados para melhoria de planos de execução, sendo que informações sobre volume de dados, distribuição e partições são fundamentais para determinação da estratégia de processamento (TrinoDB, 2025). O *DyraSQL* complementa esta abordagem através de utilização de metadados para seleção da arquitetura de processamento antes da geração do plano de execução, sendo que esta análise prévia permite direcionamento mais eficiente das consultas.

3.4 Diferenças e Contribuições do *DyraSQL*

O *DyraSQL* diferencia-se dos trabalhos relacionados através de sua abordagem de roteamento dinâmico baseado em análise prévia de metadados do *Apache Iceberg* para direcionamento de consultas entre *clusters Trino* de capacidades distintas. Enquanto trabalhos anteriores focam em melhoria dentro de uma única arquitetura (WEINTRAUB *et al.*, 2024) ou seleção entre diferentes *engines* (STRAUSZ; PARDON; GIURGIU, 2025), o *DyraSQL* propõe roteamento entre níveis de infraestrutura (*small*, *medium* e *large*) baseado em características de cada consulta identificadas através de metadados.

A principal contribuição do *DyraSQL* reside na combinação de análise prévia de metadados, algoritmo de decisão baseado em pontuação e sistema de histórico para roteamento dinâmico que melhora a relação entre custo e *performance*, complementando trabalhos que focam em melhoria durante a execução (ZHANG; ZHANG; MENG, 2025) ou seleção entre *engines* (STRAUSZ; PARDON; GIURGIU, 2025) com uma camada adicional de seleção prévia do *cluster*. Os experimentos em ambiente local demonstram que essa abordagem resulta em classificação consistente das consultas.

4 METODOLOGIA

A metodologia proposta fundamenta-se na análise sistemática de metadados de tabelas *Apache Iceberg* para tomada de decisões sobre o ambiente de execução mais adequado, resultando em melhor custo-benefício no processamento. O sistema proposto, denominado *DyraSQL*, implementa roteamento dinâmico de consultas SQL baseado em metadados.

4.1 Arquitetura da Solução Proposta

O *DyraSQL* implementa uma arquitetura que integra múltiplos componentes para análise de consultas e direcionamento automático para *clusters* com capacidades distintas. A arquitetura é composta por quatro componentes principais que trabalham de forma integrada, direcionando consultas leves para *clusters* menores e consultas pesadas para infraestrutura mais robusta. O *Query Analyzer* executa o comando *EXPLAIN (TYPE IO)* do *Trino* para extrair metadados sobre volume de dados, custos de I/O e características das consultas, incluindo tamanho estimado dos dados de saída, número estimado de linhas, custo de CPU e informações sobre filtros aplicados. O *Decision Engine* aplica o algoritmo de pontuação baseado nos metadados extraídos para determinar o ambiente de execução mais adequado (*small*, *medium* ou *large*). O *Trino Gateway Proxy* intercepta consultas SQL e integra-se com o *DyraSQL Core* para obter decisões de roteamento, direcionando a consulta para o *cluster* selecionado através do *Trino Gateway* oficial. O *Monitoring System* coleta métricas de execução e atualiza o histórico para

aprendizado contínuo.

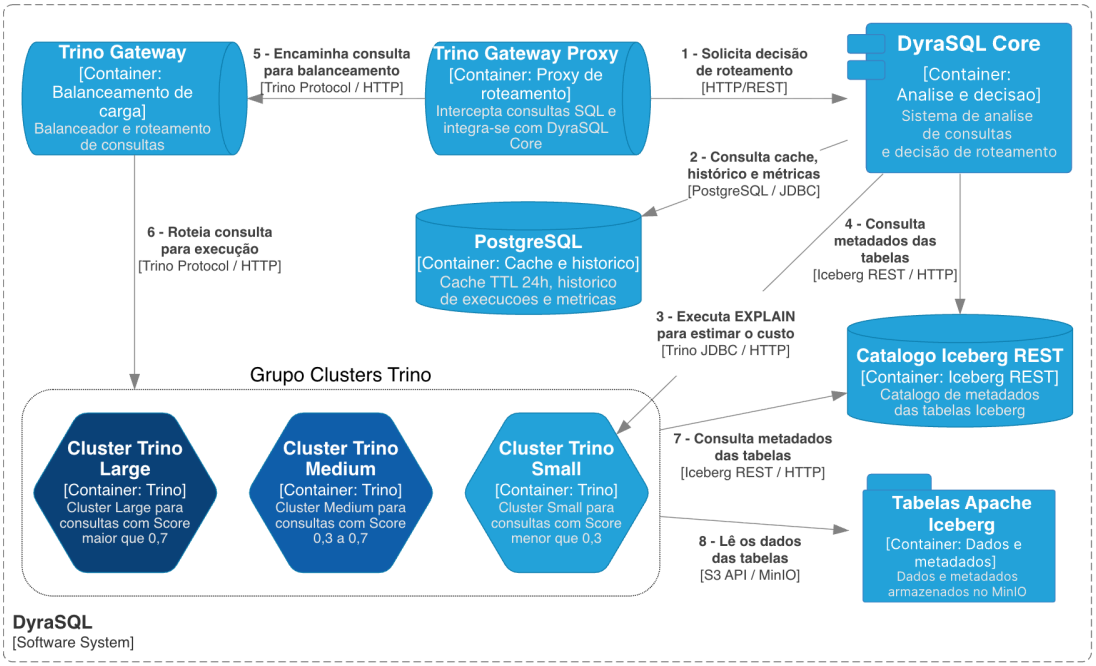
Figura 2 – Contexto do Sistema - DyraSQL



Fonte: Elaborado pelo autor.

A Figura 2 apresenta o contexto do *DyraSQL*, mostrando como o sistema se relaciona com usuários e com o ambiente de execução local. O analista de dados executa consultas SQL que são processadas pelo *DyraSQL*, que utiliza *clusters Trino* e o *data lake* local para roteamento.

Figura 3 – Containers do Sistema - *DyraSQL*

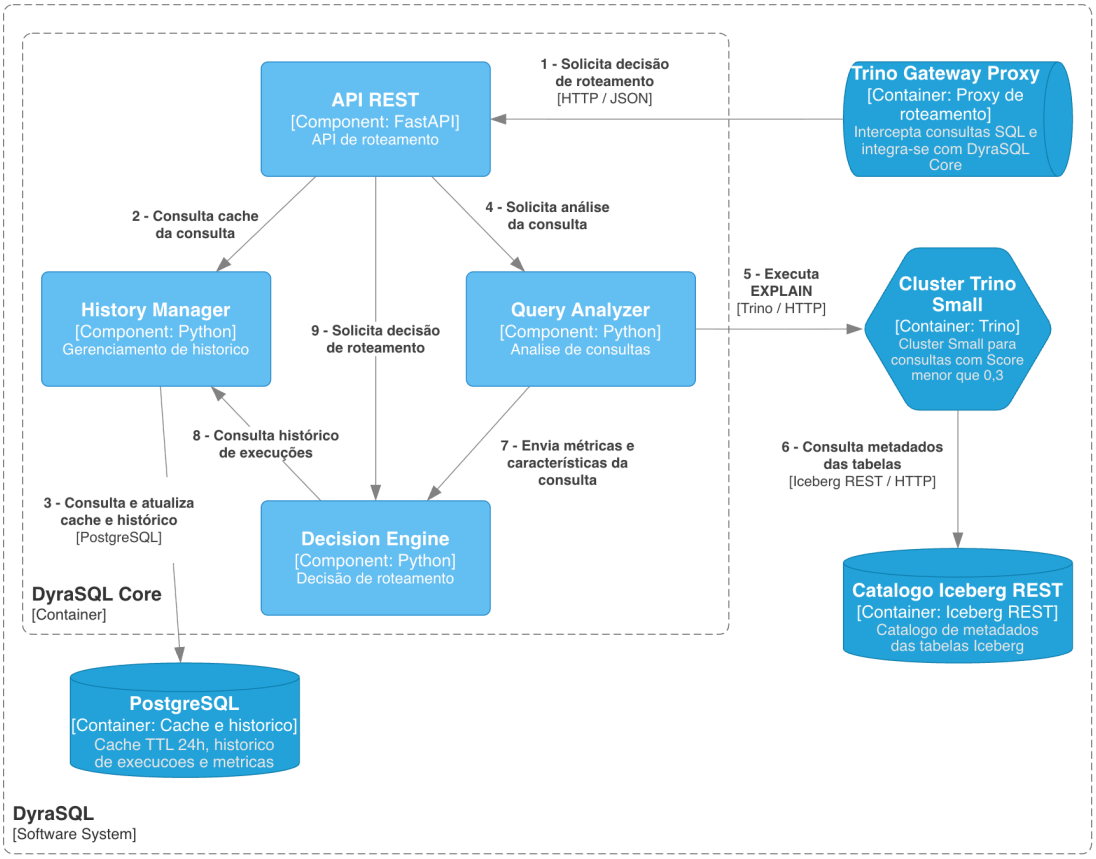


Fonte: Elaborado pelo autor.

A Figura 3 detalha a arquitetura de containers do *DyraSQL*, evidenciando os principais componentes, sendo que o *Trino Gateway Proxy* é responsável pela interceptação de consultas e integração com o *DyraSQL Core*, o *Trino Gateway* oficial atua como balanceador, o *DyraSQL Core* realiza a análise de consultas utilizando *EXPLAIN (TYPE IO)* e a decisão de roteamento, e os *clusters* de execução realizam o processamento. A integração com PostgreSQL permite o armazenamento de histórico e *cache* de decisões, enquanto o acesso aos metadados é realizado através do comando *EXPLAIN (TYPE IO)* do *Trino* sobre tabelas no MinIO. Os três *containers*

Trino representados na Figura 3, denominados *cluster large*, *cluster medium* e *cluster small*, correspondem às três faixas de capacidade descritas na Seção 4.4, sendo cada um configurado com limites distintos de memória e paralelismo dentro do ambiente Docker Compose e acessando igualmente o mesmo Catálogo Iceberg REST, o que garante consistência dos dados independentemente do *container* selecionado.

Figura 4 – Componentes do *DyraSQL* Core



Fonte: Elaborado pelo autor.

A Figura 4 apresenta os componentes internos do *DyraSQL Core*, demonstrando o fluxo de dados desde a interceptação da consulta até a execução no *cluster* selecionado e o *feedback* para o histórico. A arquitetura evidencia a integração entre os componentes do sistema, sendo que cada módulo contribui para o processo de tomada de decisão baseado em metadados.

4.2 Extração de Metadados Relevantes

O *DyraSQL* utiliza o comando *EXPLAIN (TYPE IO)* do *Trino* para extrair informações sobre o volume de dados e custos de I/O que serão processados por cada consulta SQL. Esta abordagem aproveita a capacidade do *Trino* de analisar consultas antes da execução, fornecendo estimativas baseadas no plano de execução. O *EXPLAIN (TYPE IO)* retorna informações estruturadas em formato *JSON* contendo os metadados necessários para o Algoritmo de decisão,

sendo que o sistema extrai especificamente *outputSizeInBytes*, tamanho estimado dos dados de saída em *bytes*, já considerando filtros aplicados; *outputRowCount*, número estimado de linhas; *cpuCost*, custo estimado de processamento; e *columnConstraints*, informações sobre filtros aplicados nas colunas. A Tabela 1 apresenta as siglas utilizadas nos cálculos do Algoritmo de decisão.

Tabela 1 – Siglas Utilizadas nos Cálculos do Algoritmo de Decisão

Sigla	Descrição
S	*Score* final do Algoritmo de decisão
f_v	Fator de volume normalizado
f_c	Fator de complexidade normalizado
f_h	Fator histórico normalizado
w_1	Peso do Fator volume
w_2	Peso do Fator complexidade
w_3	Peso do Fator histórico
T_e	Tamanho efetivo em GB (obtido de *outputSizeInBytes*)
R_e	Número de linhas efetivas (obtido de *outputRowCount*)
C_e	Custo de CPU efetivo (obtido de *cpuCost*)
T_{max}	Volume máximo do *warehouse* em GB (configurável por ambiente)
R_{max}	Cardinalidade máxima do *warehouse* em linhas (configurável por ambiente)
F_o	Fator de ajuste
J	Número de JOINs
Ag	Número de agregações
S_q	Número de subconsultas
F_p	Número de filtros particionados
F_{np}	Número de filtros não-particionados
L_c	Teto de normalização do Fator complexidade (configurável por ambiente)
S_p	*Score* prévio da mesma consulta

Fonte: Elaborado pelo autor.

A Tabela 1 apresenta as variáveis utilizadas no Algoritmo de decisão, organizadas desde o *Score* final até os parâmetros específicos de cada fator. Esta padronização uniformiza a notação matemática utilizada nas fórmulas do Algoritmo, facilitando a compreensão dos cálculos nas seções subsequentes. Com essas variáveis definidas, é possível demonstrar a aplicação prática da metodologia através de um exemplo concreto de consulta SQL.

4.3 Consulta SQL de Referência

Para demonstrar a aplicação prática da metodologia, considera-se a consulta SQL apresentada no Algoritmo 1, que processa a Tabela 2. A escolha dessa consulta como referência não é arbitrária, sendo que sua estrutura combina, deliberadamente, um volume de dados moderado, filtro por coluna particionada e complexidade estrutural intermediária, de modo que os três fatores do Algoritmo de decisão, volume, complexidade e histórico, contribuam de forma

perceptível para o *Score* final. Consultas mais simples, sem *JOIN* ou agregações, tenderiam a produzir Fator de complexidade próximo de zero, mascarando a contribuição relativa do Fator volume no cálculo, o que motivou a seleção de uma consulta com complexidade intermediária para fins de demonstração.

Tabela 2 – Dataset de Teste

Característica	**Descrição**
Formato	*Apache Iceberg*
Armazenamento	MinIO
Tamanho aproximado	Até 10 GB
Particionamento	Coluna *date* (função *day*)
Origem	Tabela sintética gerada no ambiente local
Colunas	**Tipo**
tenant_id	string
status	string
valor	double
date	timestamp
created_at	timestamp
updated_at	timestamp
document	string (*JSON*)
surrogate_key	string

Fonte: Elaborado pelo autor.

A Tabela 2 apresenta as características do *dataset* utilizado para demonstração da metodologia proposta, sendo que o formato *Apache Iceberg* permite particionamento por *date*, facilitando a aplicação de filtros que reduzem o volume efetivo de dados processados. O armazenamento no MinIO e o tamanho de até 10 GB representam um cenário controlado para execução local, dimensionado para caber na memória disponível da máquina utilizada nos experimentos, sendo que o particionamento pela função *day* aplicada à coluna *date* permite que o sistema processe apenas frações relevantes dos dados através de filtros temporais. Os registros foram gerados por um script de dados sintéticos, executado continuamente por um período determinado, simulando o crescimento incremental de um *data lake* em produção.

Algoritmo 1 - Consulta de Análise de Dados

```
SELECT t.tenant_id, t.categoria,
       COUNT(*) AS total_registros,
       SUM(d.valor) AS valor_total,
       AVG(d.valor) AS valor_medio
FROM dados d
JOIN tenant_info t ON d.tenant_id = t.tenant_id
WHERE d.date >= '2024-01-01'
  AND d.date < '2024-02-01'
```

```
  AND d.status = 'ATIVO'
  AND t.categoria IN (
        SELECT categoria
        FROM tenant_info
        WHERE nivel > (SELECT AVG(nivel) FROM tenant_info)
      )
GROUP BY t.tenant_id, t.categoria
HAVING COUNT(*) > 100
ORDER BY valor_total DESC
LIMIT 50;
```

Fonte: Elaborado pelo autor.

Esta consulta possui 1 JOIN com tabela de dimensão (tenant_info) utilizando a chave tenant_id, 3 agregações (COUNT, SUM, AVG), 2 subconsultas e filtros por data e status. O sistema analisa a complexidade estrutural da consulta, identificando operações que requerem processamento distribuído. O filtro por data na coluna particionada permite que o sistema processe apenas uma fração dos dados armazenados, sendo que o *range* da cláusula WHERE determina a quantidade efetiva de dados a serem lidos.

4.4 Algoritmo de Decisão de Roteamento

O Algoritmo de decisão implementa uma abordagem baseada em pontuação que combina metadados extraídos com características da consulta SQL. O processo opera em duas fases principais, sendo que a primeira é a verificação de *cache*, que consulta o PostgreSQL para verificar se existe uma decisão recente para a mesma *query fingerprint*, e a segunda é o cálculo dinâmico, que executa análise completa dos metadados quando não há *cache* válido. O *fingerprint* é obtido normalizando a consulta SQL original (letras minúsculas, espaços colapsados e literais substituídos por um marcador genérico) e aplicando a função *hash* SHA-256 sobre o resultado, de modo que consultas estruturalmente idênticas com valores de filtro distintos produzam o mesmo *fingerprint* e reaproveitem o *cache*.

O *DyraSQL* mantém um *cache* temporal no PostgreSQL com *TTL* (*Time To Live*) de 24 horas, considerando que o volume de dados pode crescer através de ingestões. Para consultas com *fingerprint* similar executadas nas últimas 24 horas, o sistema recupera diretamente a decisão de roteamento armazenada, eliminando a necessidade de re-análise dos metadados. Quando o *cache* expira ou não existe, o sistema executa o comando *EXPLAIN (TYPE IO)* no *Trino* para obter informações sobre o volume de dados e custos de I/O. O Algoritmo então considera três fatores para determinar a arquitetura mais adequada, sendo que o primeiro é o volume de dados, baseado no campo *outputSizeInBytes* retornado pelo *EXPLAIN (TYPE IO)*, o segundo é a complexidade da consulta, analisando número de JOINs, agregações e subconsultas, e o terceiro é

o histórico de execução, utilizando padrões de consultas similares. A Pontuação é calculada através da Equação 1.

$$S = w_1 \times f_v + w_2 \times f_c + w_3 \times f_h \tag{1}$$

Com base no *Score* calculado, as consultas são direcionadas para diferentes arquiteturas, sendo que *Scores* menores que 0,3 são direcionados para o *cluster small*, *Scores* entre 0,3 e 0,7 para o *cluster medium*, e *Scores* maiores que 0,7 para o *cluster large*. O sistema pode ajustar os pesos baseado no histórico de custo e *performance* das decisões tomadas, aprendendo qual arquitetura oferece melhor relação entre custo e *performance* para cada tipo de consulta.

4.4.1 Exemplo Prático de Cálculo

Para demonstrar o funcionamento do Algoritmo, considera-se o Algoritmo 1 executado sobre a Tabela 2. O sistema executa *EXPLAIN (TYPE IO)* no *Trino*, que retorna informações sobre o volume efetivo de dados a serem processados, já considerando os filtros aplicados. Para este exemplo, o *EXPLAIN (TYPE IO)* retorna *outputSizeInBytes* de aproximadamente 1,25 GB (1.342.177.280 *bytes*) e *outputRowCount* de aproximadamente 2.500.000 linhas, indicando que apenas uma fração dos dados seria processada devido ao filtro por *date* na coluna particionada. No ambiente local o volume absoluto é menor, porém a mesma normalização se aplica, sendo que T_{max} e R_{max} representam o volume máximo do *warehouse* do ambiente em que o sistema é implantado.

O Fator volume é calculado através da Equação 2, sendo que o sistema utiliza os dados retornados pelo *EXPLAIN (TYPE IO)*, normalizando separadamente o tamanho em GB e o número de linhas usando logaritmos, e combina os valores normalizados com pesos de 0,6 para tamanho e 0,4 para linhas, aplicando um fator de 0,1.

$$f_v = \left(\frac{\log(T_e)}{\log(T_{max})} \times 0,6 + \frac{\log(R_e)}{\log(R_{max})} \times 0,4 \right) \times (1 - F_o) \tag{2}$$

Os valores 1000 GB e 10^9 linhas utilizados no exemplo representam o volume máximo do *warehouse* considerado nessa configuração, e não o tamanho do *dataset* local, limitado a 10 GB na Tabela 2. A normalização logarítmica da Equação 2 compara o volume efetivo da consulta com o teto do próprio *warehouse*, de modo que T_{max} e R_{max} são parâmetros de implantação, ajustáveis a cada ambiente. No protótipo local, esses tetos seriam calibrados para cerca de 10 GB; no exemplo numérico, adota-se 1 TB e 10^9 linhas como o volume máximo de um *warehouse* de produção, na escala de *lakehouses* discutida por Xue et al. (2024) (XUE *et al.*, 2024) e coerente com a densidade do exemplo, em que 1,25 GB correspondem a aproximadamente $2,5 \times 10^6$ linhas.

Para o exemplo, o *EXPLAIN (TYPE IO)* retorna tamanho de 1,25 GB ($T_e = 1,25$) e aproximadamente 2.500.000 linhas ($R_e = 2.500.000$), com $T_{max} = 1000$ GB, $R_{max} = 10^9$ e

$F_o = 0,1$, resultando em normalização de tamanho de 0,03 e normalização de linhas de 0,71, calculando o Fator volume através da Equação 3.

$$f_v = \left(\frac{\log(1,25)}{\log(1000)} \times 0,6 + \frac{\log(2500000)}{\log(10^9)} \times 0,4\right) \times 0,9 = (0,03 \times 0,6 + 0,71 \times 0,4) \times 0,9 = 0,27 \tag{3}$$

A análise estrutural da consulta é realizada por meio da biblioteca *sqlglot*, que converte o texto SQL em uma árvore de sintaxe abstrata, permitindo identificar precisamente o número de *JOINs*, as agregações da lista de projeção do *SELECT* principal, as subconsultas e os filtros do *WHERE* de nível superior agrupados pela coluna referenciada, evitando ambiguidades de uma análise baseada em expressões regulares (como contabilizar duplamente uma agregação repetida em *HAVING*). Para a consulta do Algoritmo 1, essa análise identifica 1 JOIN, 3 agregações, 2 subconsultas, 1 filtro particionado e 2 não-particionados. Este cálculo considera a complexidade das operações SQL para determinar o nível de recursos computacionais necessários, sendo calculado através da Equação 4.

$$f_c = \frac{1 \times 0.2 + 3 \times 0.15 + 2 \times 0.25 + 1 \times 0.02 + 2 \times 0.1}{L_c} = \frac{1,37}{2,0} = 0,69 \tag{4}$$

A soma ponderada dos elementos estruturais é dividida pelo Limite de complexidade para que o Fator complexidade permaneça no intervalo $[0,1]$, saturando quando a consulta atinge ou ultrapassa esse teto. Os coeficientes, como 0,2 por *JOIN* e 0,25 por subconsulta, não derivam de um estudo estatístico sobre custo real de execução e são configuráveis por ambiente. O valor $L_c = 2$ foi escolhido em função da própria consulta de referência, cuja soma ponderada é $1,37$. Com teto 1, essa consulta saturaria em $f_c = 1$ e consultas mais pesadas deixariam de ser distinguíveis. Com teto muito maior, o Fator complexidade perderia influência no *Score*. O inteiro 2 é o teto imediatamente acima de $1,37$, deixa folga para consultas com mais *JOINs* e subconsultas, e produz $f_c = 0,69$ no exemplo, na região intermediária da escala.

O Fator histórico é calculado com base em execuções anteriores da mesma consulta (mesmo *fingerprint*), conforme a Tabela 3. Sem histórico, o sistema utiliza o valor *default* $0,5$, que representa neutralidade. O exemplo encontra-se nessa condição.

Tabela 3 – Cálculo do Fator Histórico

Condição	**Valor de f_h**
Sem histórico disponível	$0,5$
Execução anterior bem-sucedida	S_p
Falha com $S_p \leq 0,7$ (*small* ou *medium*)	$1 - S_p$
Falha com $S_p > 0,7$ (*large*)	$0,5$

Fonte: Elaborado pelo autor.

A operação $1 - S_p$ aplica-se apenas quando a falha ocorreu em *cluster small* ou *medium*,

interpretada como possível subdimensionamento, e então reflete o Score em torno de $0,5$ para puxar a próxima decisão a uma faixa de maior capacidade. Falha no *cluster large* não autoriza reduzir a capacidade, pois o ambiente mais forte já foi utilizado e a causa tende a ser outra, como erro da consulta, dados ou *timeout* alheio ao provisionamento. Nesse caso o Fator histórico permanece neutro $(0,5)$, deixando volume e complexidade definirem o Score. Com os pesos dos testes ($w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.2$) e $f_h = 0,5$, o Score final é calculado através da Equação 5.

$$S = 0.5 \times 0.27 + 0.3 \times 0.69 + 0.2 \times 0.5 = 0.44 \tag{5}$$

Com *Score* de 0,44, a consulta é direcionada para o *cluster medium* (faixa 0,3 a 0,7). Os pesos podem ser ajustados com base no histórico de custo e *performance* das decisões, inspirado em sistemas como o Amazon Redshift MDDL (DING *et al.*, 2024). O Algoritmo principal do *DyraSQL* pode ser expresso através do pseudocódigo apresentado no Algoritmo 2, sendo que este processo garante que cada consulta seja direcionada para o ambiente mais adequado baseado na análise dos metadados obtidos através do *EXPLAIN (TYPE IO)* do *Trino*.

Algoritmo 2 - Roteamento Dinâmico de Consultas

```
Entrada: consulta, acesso ao mecanismo de consulta (plano de
         execução), banco de cache e histórico
Saída: cluster selecionado, pontuação (score) calculada

1. Gerar fingerprint único da consulta
2. Consultar o cache de decisões usando o fingerprint
3. Se a decisão existir no cache e o TTL for válido, então
      Retornar o cluster armazenado
   Fim se
4. Obter o plano de execução e os metadados junto ao
   mecanismo de consulta
5. Extrair métricas de volume (tamanho estimado de saída,
   contagem de linhas e restrições de colunas)
6. Normalizar o fator de volume com base no tamanho de
   saída estimado
7. Analisar a complexidade da consulta
8. Consultar o histórico por consultas similares
9. Calcular a pontuação utilizando a fórmula ponderada
10. Se a pontuação for menor que 0.3, então
       Selecionar o cluster pequeno (small)
    Senão se a pontuação estiver entre 0.3 e 0.7, então
       Selecionar o cluster médio (medium)
    Senão
       Selecionar o cluster grande (large)
    Fim se
```

```
11. Salvar a decisão e as métricas no histórico de roteamento
12. Retornar o cluster selecionado e a pontuação calculada
```

Fonte: Elaborado pelo autor.

O pseudocódigo demonstra o fluxo completo do Algoritmo, desde a interceptação da consulta até a decisão final de roteamento, incluindo verificação de *cache*, execução do *EXPLAIN (TYPE IO)* para extração de metadados, cálculo do *Score* e seleção do *cluster* apropriado.

4.5 Sistema de Histórico e Aprendizado

O *DyraSQL* mantém um histórico de execuções no PostgreSQL, armazenando *fingerprints* de consultas, métricas de custo-benefício e decisões de roteamento. A cada execução de consulta, o sistema pode coletar métricas pós-execução incluindo tempo de execução, custo operacional e eficácia da decisão, salvando essas informações para enriquecimento do histórico. A eficácia é avaliada pela comparação entre o tempo observado e o esperado para a faixa de capacidade, sendo que decisões recorrentemente inadequadas podem indicar a necessidade de ajuste dos pesos w_1, w_2 e w_3 da Equação 1.

4.5.1 Cálculo do Fator Histórico

O Fator histórico é calculado através da análise do histórico de execuções anteriores da mesma consulta, identificada pelo *fingerprint* único gerado a partir da estrutura da consulta SQL. O sistema consulta o PostgreSQL para verificar se existe uma execução anterior da mesma consulta, sendo que quando não há histórico disponível, o sistema utiliza um valor *default* de $f_h = 0,5$, representando neutralidade na decisão.

Quando há histórico disponível, o sistema verifica se a execução anterior foi bem-sucedida. Em caso de sucesso, o Fator histórico reutiliza o Score prévio, reforçando a faixa de capacidade que funcionou. Em caso de falha em *cluster small* ou *medium*, utiliza o complemento $1 - S_p$ para buscar mais capacidade. Em caso de falha no *cluster large*, o Fator histórico permanece em $0,5$, porque repetir a consulta em *clusters* menores não corrige um problema que já ocorreu no ambiente mais forte e não indica, por si só, déficit de *performance*.

O Fator histórico é então determinado conforme a Tabela 3, sendo que esta abordagem permite que o sistema aprenda com execuções anteriores da mesma consulta. Esta abordagem é inspirada em mecanismos de *learned cost models* implementados em sistemas como o Amazon Redshift (DING *et al.*, 2024). O sistema mantém um *cache* temporal no PostgreSQL com

TTL de 24 horas, sendo que consultas com *fingerprint* similar executadas nas últimas 24 horas recuperam diretamente a decisão de roteamento armazenada.

4.6 Integração com *Trino Gateway*

A solução integra-se ao *Trino* Gateway oficial através de uma arquitetura modular composta por três componentes principais. O primeiro componente é o *Trino* Gateway oficial, compilado a partir do código-fonte em Java, que atua como balanceador e *gateway* de roteamento, utilizando PostgreSQL para persistência de configurações e histórico de *backends*. O segundo componente é o *Trino Gateway Proxy*, desenvolvido em Python utilizando FastAPI, que intercepta consultas SQL antes do roteamento e integra-se com o *DyraSQL Core* para obter decisões baseadas em análise de metadados. O terceiro componente é o *DyraSQL Core*, que implementa o Algoritmo de decisão e fornece a API REST para análise de consultas e cálculo de *scores*. O *Trino Gateway Proxy* consulta o *DyraSQL Core* através do *endpoint* POST /api/v1/route para cada consulta recebida, obtendo a decisão de roteamento com *cluster* recomendado, *score* calculado e fatores utilizados, e então direciona a consulta para o *cluster Trino* apropriado. Esta arquitetura modular permite integração com o *Trino* Gateway oficial sem necessidade de modificações no código-fonte do *gateway*.

O contrato do *endpoint* POST /api/v1/route recebe um objeto *JSON* com a consulta SQL e retorna a decisão de roteamento (*cluster*, *score* e fatores utilizados), conforme exemplo de requisição e resposta apresentado no Apêndice A.

4.7 Implementação do *framework*

O *framework DyraSQL* foi desenvolvido utilizando Python para os módulos de roteamento e análise de metadados, Docker Compose para orquestração do ambiente local, e *Trino* como *engine* de consultas. A arquitetura inclui o *Trino Gateway* oficial, o *Trino Gateway Proxy* em FastAPI, PostgreSQL para persistência do *gateway* e para *cache* de decisões, MinIO para armazenamento de objetos das tabelas *Iceberg*, e catálogo REST compartilhado pelos *clusters*.

A configuração do *Trino* utiliza o conector *Iceberg* com catálogo REST e *filesystem* de objetos apontando para o MinIO. Esta configuração garante que os três *clusters* compartilhem o mesmo catálogo de metadados, permitindo acesso unificado às tabelas independentemente do *cluster* de execução. A implementação em Python contempla módulos para análise de consultas SQL utilizando *EXPLAIN (TYPE IO)*, aplicação do Algoritmo de decisão baseado em *scores*, gerenciamento de histórico no PostgreSQL e API REST para integração com o *Trino Gateway Proxy*. O *framework* foi avaliado através de testes com tabelas *Apache Iceberg* geradas no ambiente local, confirmando que o sistema é capaz de calcular *scores* e direcionar consultas para os *clusters* apropriados. O código-fonte do protótipo está disponível no Apêndice B.

5 EXPERIMENTOS E RESULTADOS

Esta seção apresenta o ambiente empregado para avaliação do *framework DyraSQL*, os cenários de teste, as métricas observadas e a discussão dos resultados. Os experimentos foram projetados para verificar a consistência da lógica de roteamento e dos pesos do Algoritmo de decisão em ambiente local, confirmando que o sistema calcula *scores* e classifica consultas nas faixas de capacidade definidas.

5.1 Ambiente de Teste

Os experimentos foram conduzidos em ambiente local orquestrado por Docker Compose, com tabelas *Apache Iceberg* geradas sobre o MinIO e catálogo REST compartilhado pelos *clusters Trino*. A Tabela 2 foi utilizada como caso de referência, sendo que a consulta principal foi apresentada através do Algoritmo 1. Experimentos adicionais variaram o *range* de filtros por data para processar diferentes volumes efetivos.

A Tabela 4 apresenta as configurações do ambiente de teste. O *framework* direciona consultas para três *clusters* baseado no *Score* calculado, sendo que consultas com *Score* menor que 0,3 seguem para o *cluster small*, consultas com *Score* entre 0,3 e 0,7 para o *cluster medium*, e consultas com *Score* maior que 0,7 para o *cluster large*.

Tabela 4 – Configurações do Ambiente de Teste

Componente	**Tipo**	**Configuração**
cluster small	*Small*	*Score* menor que 0,3
cluster medium	*Medium*	*Score* 0,3 a 0,7
cluster large	*Large*	*Score* maior que 0,7
Trino Gateway	*gateway*	Balanceador e roteamento
Trino Gateway Proxy	*proxy*	Integração com o *DyraSQL Core*
DyraSQL Core	API REST	Análise e decisão de roteamento
PostgreSQL	Banco de dados	Persistência do *gateway* e *cache*
MinIO	Objetos	*Warehouse* das tabelas *Iceberg*
Catálogo REST	Catálogo	Metadados das tabelas *Iceberg*

Fonte: Elaborado pelo autor.

A Tabela 4 reúne os componentes da stack local, evidenciando a separação entre decisão de roteamento, catálogo, armazenamento e execução. Os três *clusters Trino* compartilham o mesmo catálogo REST e o mesmo *warehouse* no MinIO, de modo que a escolha do destino altera apenas a capacidade de processamento.

5.2 Cenários de Teste

Os testes utilizaram a Tabela 2 e a consulta do Algoritmo 1, além de variações com filtros de data mais estreitos ou mais amplos e de consultas apenas de metadados. Os cenários cobriram *cache miss*, primeira execução com cálculo completo do *Score*, e *cache hit*, reexecução com o mesmo *fingerprint* dentro do *TTL* de 24 horas. Também foram observadas consultas de baixa complexidade, encaminhadas ao *cluster small*, e a consulta de referência, cujo exemplo de cálculo resulta em *Score* 0,44 e destino *medium*.

5.3 Métricas e Resultados

As métricas observadas incluem consistência dos *scores* para consultas similares, adequação das faixas 0,3 e 0,7, tempo de cálculo associado ao *EXPLAIN (TYPE IO)* e taxa de *cache hit* no PostgreSQL. O *EXPLAIN (TYPE IO)* retornou estimativas de volume e de linhas compatíveis com os filtros de partição aplicados, permitindo o cálculo do Fator volume. A consulta de referência produziu *Score* 0,44, classificada no *cluster medium*, enquanto consultas sem junções e com filtros estreitos tenderam ao *cluster small*. Reexecuções com o mesmo *fingerprint* recuperaram a decisão armazenada, evitando nova análise enquanto o *TTL* permaneceu válido.

Os pesos $w_1 = 0,5$, $w_2 = 0,3$ e $w_3 = 0,2$ diferenciaram consultas leves, intermediárias e pesadas no conjunto avaliado, sendo o fator de volume o de maior influência e o fator histórico complementar quando havia execução prévia. O tempo de análise prévia manteve-se adequado para o *dataset* sintético, permanecendo na ordem de centenas de milissegundos nas consultas observadas.

5.4 Limitações Identificadas

O *dataset* sintético possui volume inferior ao de ambientes de produção, o que restringe a generalização dos resultados para *warehouses* de centenas de gigabytes. A precisão das decisões depende de histórico suficiente no PostgreSQL, sendo que consultas novas utilizam $f_h = 0,5$. O indicador de sucesso da execução é binário e não classifica a causa da falha, de modo que uma falha em *cluster small* ou *medium* ainda pode dever-se a erro de SQL e não a subdimensionamento. O *EXPLAIN (TYPE IO)* pode se tornar mais lento em planos muito complexos, e metadados *Iceberg* desatualizados reduzem a qualidade das estimativas. A máquina local também não reproduz elasticidade nem escala de um ambiente de nuvem, aspecto tratado nos trabalhos futuros. O sistema ainda não implementa uma estratégia formal de *fallback* para os casos em que o *EXPLAIN (TYPE IO)* não retorna resultado (indisponibilidade do *Trino* ou *timeout*), resultando em erro em vez de um *cluster* padrão; essa condição não se manifestou nos testes, mas não é coberta pela lógica atual.

6 CONSIDERAÇÕES FINAIS

Este trabalho apresentou o desenvolvimento do *DyraSQL*, um mecanismo de decisão para roteamento dinâmico de consultas SQL baseado em metadados de tabelas *Apache Iceberg*. O problema abordado consistiu na necessidade de arquiteturas que se adaptem ao tipo de consulta, evitando provisionamento excessivo de infraestrutura robusta para consultas leves e garantindo recursos adequados para consultas pesadas, resultando em melhor custo-benefício no processamento.

Os objetivos propostos foram alcançados através da análise de metadados *Apache Iceberg*, estabelecimento de regras de decisão para roteamento, desenvolvimento de um *framework* funcional em ambiente local e avaliação da lógica de roteamento. A metodologia envolveu análise dos metadados disponíveis, definição de critérios baseados em volume de dados, complexidade da consulta e histórico de execuções, e verificação da lógica proposta através de testes com tabelas *Iceberg* geradas localmente.

Os principais resultados demonstraram que a lógica de roteamento funciona de forma consistente, sendo que o *framework* é capaz de calcular *scores* e direcionar consultas para os *clusters* apropriados baseado nas características dos dados e na complexidade das consultas. A análise dos metadados revelou informações sobre volume de dados e complexidade que podem ser utilizadas para determinar a arquitetura mais adequada, sendo que o uso do *EXPLAIN (TYPE IO)* do *Trino* fornece estimativas suficientes para o Algoritmo de decisão. A avaliação confirmou que os pesos do Algoritmo funcionam de forma adequada, sendo que a combinação dos fatores de volume, complexidade e histórico resulta em *scores* que diferenciam consultas leves, intermediárias e pesadas.

As contribuições deste trabalho incluem um *framework* para análise de metadados *Apache Iceberg* para determinação de arquiteturas dinâmicas baseado em volume de dados, complexidade de consultas e histórico de execuções; regras de decisão para roteamento automático através de algoritmo de pontuação que combina múltiplos fatores; um protótipo funcional integrando *Trino*, catálogo *Iceberg* REST, MinIO e PostgreSQL, que direciona consultas para *clusters small*, *medium* ou *large*; e a avaliação da lógica de roteamento através de testes com dados sintéticos em ambiente local.

Como limitações do estudo, destacam-se o uso de *dataset* sintético em escala inferior à de produção e a dependência de histórico suficiente no PostgreSQL, sendo que consultas sem histórico apresentam maior probabilidade de decisões menos precisas. A extração de metadados apresenta limitações para consultas muito complexas, sendo que o tempo do *EXPLAIN (TYPE IO)* pode impactar a latência total. O sistema também depende da qualidade dos metadados do *Apache Iceberg*, sendo que tabelas com metadados desatualizados podem resultar em estimativas imprecisas do volume efetivo.

6.1 Trabalhos Futuros

Trabalhos futuros concentram-se na aplicação do *DyraSQL* em ambientes de nuvem. O núcleo do sistema, composto pelo cálculo do *Score*, pelo *Trino Gateway* e pelas três faixas de *cluster*, permanece o mesmo, sendo que o que se altera é a implantação. *Clusters* gerenciados em nuvem, por exemplo EMR (Amazon Web Services, 2025b), e o armazenamento de objetos do provedor substituiriam o MinIO e as instâncias *Trino* locais, permitindo avaliar o roteamento com volumes de produção e com elasticidade típica de plataformas gerenciadas.

Outra direção consiste em substituir os pesos w_1, w_2 e w_3, atualmente fixos, por um modelo de aprendizado supervisionado treinado sobre o histórico acumulado no PostgreSQL, de forma similar ao Amazon Redshift MDDL (DING *et al.*, 2024), além de avaliar o *DyraSQL* sobre tabelas de centenas de gigabytes a terabytes, verificando se as faixas de *score* permanecem adequadas em escala real.

Uma limitação da arquitetura atual é que os três *clusters* são *containers* persistentes com capacidade fixa, de modo que um pico de consultas concorrentes na mesma faixa de *score* pode esgotar o *cluster* de destino mesmo com os demais subutilizados. Um trabalho futuro é migrar para um modelo orientado a *tasks*, análogo à distinção que o Amazon ECS estabelece entre *services* de longa duração e *tasks* avulsas sob demanda (Amazon Web Services, 2025a), dimensionando o ambiente de execução individualmente por consulta em vez de compartilhar um *cluster* fixo. Também é necessário definir uma estratégia de *fallback* para os casos em que a análise de metadados não é concluída (indisponibilidade do *Trino* ou *timeout*), direcionando a um *cluster* padrão em vez de retornar erro ao cliente - situação não observada nos testes, mas esperada antes de uma implantação em produção.

Outra possibilidade consiste em analisar estatisticamente o histórico de roteamento para caracterizar o perfil de utilização da organização, identificando horários, dias e tipos de consulta em que o *cluster large* é de fato demandado. A partir desse perfil, modelos de previsão de demanda, por exemplo com séries temporais, poderiam antecipar a inicialização desse ambiente nos intervalos de maior carga e desligá-lo nos períodos de baixa utilização. Tal abordagem reduziria o custo operacional sem mantê-lo disponível continuamente e diminuiria a latência percebida pelas consultas pesadas, pois o acionamento ocorreria antes da chegada da consulta, e não somente após ela.

REFERÊNCIAS

Amazon Web Services. **Amazon Elastic Container Service**: What is amazon ecs? 2025. Disponível em: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html>. Acesso em: 22 ago. 2026.

Amazon Web Services. **Amazon EMR - Elastic MapReduce**: What is amazon emr? 2025. Disponível em: <https://docs.aws.amazon.com/emr/latest/ManagementGuide/emr-what-is-emr.html>. Acesso em: 30 out. 2025.

Apache Software Foundation. **Apache Iceberg Table Spec**. 2025. Licensed under the Apache License, Version 2.0. Disponível em: <https://iceberg.apache.org/spec/>. Acessado em: 20 set. 2025. Disponível em: <https://iceberg.apache.org/spec/>.

DING, J. *et al.* Automated multidimensional data layouts in amazon redshift. **Amazon Science**, 2024. Disponível em: <https://www.amazon.science/publications/automated-multidimensional-data-layouts-in-amazon-redshift>. Acesso em: 24 set. 2025.

DREMIO. **Data Lake vs Data Warehouse**. 2025. Disponível em: <https://www.dremio.com/resources/guides/intro-data-lake-vs-data-warehouse/>. Acesso em: 30 out. 2025.

IBM. **Big Data Analytics**. 2025. Disponível em: <https://www.ibm.com/br-pt/think/topics/big-data-analytics>. Acesso em: 30 out. 2025.

STRAUSZ, A.; PARDON, N.; GIURGIU, I. A learned cost model-based cross-engine optimizer for sql workloads. **arXiv preprint arXiv:2506.02802**, 2025. Disponível em: <https://arxiv.org/abs/2506.02802>. Acesso em: 19 ago. 2025.

TrinoDB. **Trino Concepts**: Worker. 2025. Disponível em: <https://trino.io/docs/current/overview/concepts.html#trino-concept-worker>. Acesso em: 30 out. 2025.

TRINODB. **Trino Gateway**: A load balancer, proxy server, and configurable routing gateway for multiple trino clusters. 2025. Disponível em: <https://github.com/trinodb/trino-gateway>. Acesso em: 19 ago. 2025.

WEINTRAUB, G. *et al.* Optimizing cloud data lake queries with a balanced coverage plan. **IEEE Transactions on Cloud Computing**, v. 12, n. 1, p. 84–99, January-March 2024. Disponível em: <https://ieeexplore.ieee.org/document/10342737>. Acesso em: 04 out. 2025.

XUE, M. *et al.* Adaptive and robust query execution for lakehouses at scale. **Proceedings of the VLDB Endowment**, v. 17, n. 12, p. 3947–3959, 2024. Disponível em: <https://dl.acm.org/doi/abs/10.14778/3685800.3685818>. Acesso em: 19 ago. 2025.

ZHANG, X.; ZHANG, H.; MENG, X. Intra-query runtime elasticity for cloud-native data analysis. **arXiv preprint arXiv:2502.18184**, 2025. Disponível em: <https://arxiv.org/abs/2502.18184>. Acesso em: 25 set. 2025.

APÊNDICE A - EXEMPLO DE REQUISIÇÃO E RESPOSTA DA API DE ROTEAMENTO

Os trechos a seguir ilustram um exemplo de requisição e de resposta do *endpoint* POST /api/v1/route do *DyraSQL Core*, descrito na Seção 4.

Quadro 1 - Requisição do *endpoint* POST /api/v1/route

```
{
  "query": "SELECT ... FROM dados d JOIN tenant_info t ..."
}
```

Fonte: Elaborado pelo autor.

Quadro 2 - Resposta do *endpoint* POST /api/v1/route

```
{
  "fingerprint": "3f2a1c...",
  "cluster": "medium",
  "score": 0.44,
  "factors": {
    "volume": 0.27,
    "complexity": 0.69,
    "historical": 0.5
  },
  "cached": false,
  "cluster_url": "http://trino-medium:8080"
}
```

Fonte: Elaborado pelo autor.

APÊNDICE B - CÓDIGO-FONTE DO PROTÓTIPO

O código-fonte do *DyraSQL* está disponível em repositório público, incluindo o *DyraSQL Core*, o *Trino Gateway Proxy*, a orquestração em Docker Compose e os scripts de geração do *dataset* sintético. O endereço do repositório é apresentado no Quadro 3.

Quadro 3 - Endereço do repositório do *DyraSQL*

`https://github.com/trababalhos-puc/tcc-puc-dyrasql`

Fonte: Elaborado pelo autor.